

Deeper wrinkles

I was ready to stop. Really, I was. But the thought of risks kept gnawing at me. Finally, I gave in and hacked the example mentioned above.

Your first attempts to run should be as safe as possible. That's the motivation behind *EasyHTTPServer*. By default, it only listens on *localhost* (127.0.0.1); to access your Web server, someone has to be logged in on your machine (the machine where you invoke *EasyHTTPServer*). Once you are comfortable with what you are doing, you can tell *EasyHTTPServer* to listen to requests from beyond your machine (or you can revert to using *SimpleHTTPServer*). Here's what to do. Fetch the code below, from http://www.linuxforu.com/article_source_code/june12/5_minute_web_server.zip, and save it as *EasyHTTPServer.py* (or a name of your choice).

```
""" EasyHTTPServer.py - Not quite so Simple HTTP Server

This program enhances SimpleHTTPServer.py ever so slightly.

The user can specify a server address (server_name and server_
port),
default: localhost 8000.

Usage:

python EasyHTTPServer.py [ host_or_ipaddr port ]

host_or_ipaddr the address to listen on
can be IP address or domain name

port port number to listen on (must be an integer)

To listen on all interfaces, use

python EasyHTTPServer.py '' port

defaults to

python EasyHTTPServer.py localhost 8000
```

Based on
<http://www.linuxjournal.com/content/tech-tip-really-simple-http-server-python>

```
"""

import sys
import BaseHTTPServer
from SimpleHTTPServer import SimpleHTTPRequestHandler

HandlerClass = SimpleHTTPRequestHandler
ServerClass = BaseHTTPServer.HTTPServer
```

```
Protocol = "HTTP/1.0"

if len(sys.argv) == 1:
    server_name = '127.0.0.1'
    server_port = 8000
elif len(sys.argv) == 3:
    server_name = sys.argv[1]
    server_port = int(sys.argv[2])
else:
    sys.stderr.write("I need either no args or server_name
server_port.\n");
    sys.stderr.close()
    sys.exit(1)

server_address = (server_name, server_port)

HandlerClass.protocol_version = Protocol
httpd = ServerClass(server_address, HandlerClass)

sa = httpd.socket.getsockname()
print "Serving HTTP on", sa[0], "port", sa[1], "..."
httpd.serve_forever()
```

Find an appropriate directory—one that has the content you want to serve, and then:

```
cd directory/to/serve
python path/to/code/EasyHTTPServer.py
```

Surf to <http://localhost:8000/> with your browser. You're done! If you can't use port 8000, specify another port—for example:

```
python path/to/code/EasyHTTPServer.py localhost 8000
```



References

- [1] <http://blogs.igalia.com/eocanha/?p=109>
- [2] <http://docs.python.org/library/simplehttpserver.html>

By: Henry Grebler

The author has spent his days working with computers, mostly for computer manufacturers or software developers. His early computer experience includes relics such as punch cards, paper tape and mag tape. His darkest secret is that he has been paid to do the sorts of things he would have paid money to be allowed to do. Just don't tell any of his employers.

He has used Linux as his personal home desktop since the family got its first PC in 1996. Back then, when the family shared the one PC, it was a dual-boot Windows/Slackware setup. Now that each member has his/her own computer, Henry somehow survives in a purely Linux world. He lives in a suburb of Melbourne, Australia.